

Team Blind Drivers: Vision-Based Lane Detection and Lane-Keeping Assistance for Autonomous Vehicles

Team Members:

- Daniel Plascencia – dplascencia@cpp.edu
- Ethan J. Reidel – ejreidel@cpp.edu
- Mihail Chitorog – mchitorog@cpp.edu

Problem Statement and Background

Lane departure is a common cause of car accidents. According to Cicchino, cars with lane departure warning (LDW) systems get into 11% fewer crashes overall and 21% fewer injury crashes compared to cars without them [1]. A separate study looking at fatal lane-departure crashes in Sweden found that many of these deaths could likely have been avoided if the driver had been warned in time [2].

Lane detection has increasingly relied on computer vision and deep learning rather than older methods, which tend to struggle with poor lighting, worn lane markings, and complex road conditions. Classical, low-cost implementations have shown that lane detection can still run in real time on inexpensive hardware, using edge detection and simple tracking to steer a small vehicle prototype [3]. More recent work shows that deep learning models can also run efficiently on more capable embedded platforms: one study built a CNN-LSTM network into a full lane departure warning system running on an NVIDIA Jetson Xavier NX, reaching over 96% detection accuracy [4], while another combined lightweight lane detection with traffic sign recognition on similarly constrained hardware [5]. Broader surveys of the field confirm that CNN-based methods have become the standard approach for lane detection [6].

For our own development, we chose to use CARLA, an open-source simulator built specifically for training and testing autonomous driving systems [7]. Prior work supports CARLA as a strong fit for this kind of project: one study built a full lane-keeping pipeline inside CARLA using LaneNet for lane detection and a PID controller for steering [8], and another analyzed how camera resolution affects lane detection accuracy and processing speed within CARLA, finding that 512×256 offers a good balance between accuracy and real-time performance [9]. These findings directly inform the design of our own perception and control pipeline.

Goals and Expected Results

The primary goal of the project is to build and evaluate a vision-based lane detection and lane-keeping control system in the CARLA simulator and, if time permits, test it on physical hardware. The system should be able to detect lane boundaries from camera input, determine the vehicle's position relative to the lane center, and generate steering corrections to keep the vehicle centered within the lane across varying road geometries and driving conditions.

Methodology and Required Resources

Simulation environment. We will use CARLA as our primary development and testing platform for developing and evaluating proposed lane detection and keeping systems. CARLA provides configurable urban and highway environments, simulated vehicle models, and sensor interfaces such as RGB cameras, Depth cameras, and Wide-angle cameras, etc. All of these will enable the collection of data under controlled driving conditions. Traditional computer vision techniques using OpenCV will be used to identify lane markings and estimate lane geometry. There are also alternative methods such as deep learning approaches utilizing convolutional neural networks (CNNs) that can be explored to detect lane markings under more complex environmental conditions. CARLA's camera sensors, configurable weather, and configurable road environments will support the development of these approaches.

The detected lane information will be integrated into a lane keeping control system using CARLA's vehicle control interface on the simulated vehicle to correct for appropriate lane keeping conditions. The lane geometry extracted can be determined through computer vision or deep learning methods which will be used to estimate the vehicle's position relative to the lane and subsequently generate and execute steering commands. The system will be assessed across varying road geometries, environmental conditions, and driving scenarios using metrics such as lane detection accuracy, lateral deviation, and vehicle stability

Perception pipeline. The perception pipeline will have two phases, if time permits. The first phase will involve classical

computer vision techniques for lane perception, while the second may investigate a neural-network-based approach. For the classic component, camera photos will first be converted to grayscale images and processed using lightweight filtering for noise reduction, such as Gaussian, mean, or median filtering. Edge detection will then be performed using a method such as Canny or Sobel filtering. Similarly to the approach described in [3], we will restrict processing to only the lower half of the image, since lane boundaries relevant to the vehicle’s position are largely in this area.

In order to detect whether the vehicle is within the lines, we will use a simple horizontal scanning algorithm. Assuming the camera is approximately centered and aligned with the vehicle, we place a point centered in the bottom half of our image. We will search horizontally left and right for candidate lane boundaries. Assuming our edge detector is robust enough and we have a binary image, comparing the left and right distances will tell us if the vehicle is centered. If the difference is above a chosen threshold, the vehicle can be considered not centered in the lane. Additional filtering may be used to reject unimportant edges. Because only a small region close to the vehicle is used for the algorithm, curved boundaries can be approximated as straight lines. Other classical computer vision lane detection algorithms may be implemented and tested. If time permits, we may attempt to use a convolutional neural network (CNN) for more robust lane detection.

The classical computer vision pipeline will be implemented in Python using OpenCV, while the neural network components will be developed in Python using PyTorch.

Control pipeline. Detected lane geometry will feed a control module that computes the vehicle’s lateral offset and heading error relative to the lane center. These values will be converted into steering commands using a classical geometric or PID-based controller, allowing the vehicle to actively correct its position and remain centered within the lane. The controller will be tuned and tested across varying road curvatures and speeds to ensure stable, responsive behavior without overcorrection or oscillation. If time constraints prevent a fully tuned lane-centering controller from being completed, the system will fall back to a threshold-based departure-alert mechanism, flagging when the vehicle’s lateral deviation exceeds a defined threshold rather than actively correcting it.

Evaluation. System performance will be assessed primarily through lateral deviation from lane center, measured as the mean and maximum offset (in meters or as a percentage of lane width) across test runs, along with the root-mean-square error (RMSE) to capture overall tracking consistency. We will also measure correction responsiveness, in other words, how quickly the controller brings the vehicle back toward lane center after a deviation is introduced, and lane detection accuracy under the perception pipeline’s classical computer vision (and, if implemented, CNN) approach. Testing will be conducted across multiple CARLA maps and road geometries (straight and curved roads), under varying weather and lighting conditions, to evaluate robustness. Where feasible, results will be compared against benchmarks reported in related CARLA-based lane-keeping work (e.g., [8]) to contextualize performance.

Required resources. CARLA simulator; a CUDA-capable GPU for training; Python, with OpenCV for the classical computer vision pipeline and PyTorch for the CNN-based approach; and, for the stretch goal, an NVIDIA Jetson Nano or DRIVE AGX Orin platform.

References

- [1] J. B. Cicchino, "Effects of lane departure warning on police-reported crash rates," *Journal of Safety Research*, 2018. <https://www.sciencedirect.com/science/article/abs/pii/S002243751730556X>
- [2] "The safety potential of lane departure warning systems—A descriptive real-world study of fatal lane departure passenger car crashes in Sweden," *Traffic Injury Prevention*, 2017. <https://www.tandfonline.com/doi/full/10.1080/15389588.2017.1313413>
- [3] A. Rossi, N. Ahmed, S. Salehin, T. H. Choudhury, and G. Sarowar, "Real-time Lane Detection and Motion Planning in Raspberry Pi and Arduino for an Autonomous Vehicle Prototype," <https://arxiv.org/abs/2009.09391>
- [4] Y. Kortli, S. Gabsi, L. F. C. Lew Yan Voon, M. Jridi, M. Merzougui, and M. Atri, "Deep embedded hybrid CNN-LSTM network for lane detection on NVIDIA Jetson Xavier NX," *Knowledge-Based Systems*, vol. 240, p. 107941, 2022. <https://doi.org/10.1016/j.knosys.2021.107941>
- [5] M. T. Islam and M. R. Kabir, "Vision-Based Lane Following and Traffic Sign Recognition for Resource-Constrained Autonomous Vehicles," *arXiv:2604.22872*, 2026. <https://doi.org/10.48550/arXiv.2604.22872>
- [6] "Monocular Lane Detection Based on Deep Learning: A Survey," *arXiv:2411.16316*, 2024. <https://doi.org/10.48550/arXiv.2411.16316>
- [7] S. Malik, M. A. Khan, and H. El-Sayed, "CARLA: Car Learning to Act – An Inside Out," *Procedia Computer Science*, vol. 198, pp. 742-749, 2022. <https://doi.org/10.1016/j.procs.2021.12.316>

- [8] C. -Y. Cheng, C. -E. Shi and I. Saputra "Autonomous Lane-Keeping Using LaneNet and ROS in CARLA Simulation," 2025 IEEE International Conference on Consumer Electronics, 2025. <https://ieeexplore.ieee.org/abstract/document/11207862>
- [9] A. Csato, F. Mariasiu, and G. Csiki, "Analysis of the Influence of Image Resolution in Traffic Lane Detection Using the CARLA Simulation Environment," *Vehicles*, vol. 7, no. 2, p. 60, 2025. <https://doi.org/10.3390/vehicles7020060>